# PROJECT 6 — UNIVERSITY STUDENT INFORMATION & GUIDANCE ASSISTANT

*A campus-wide AI guide that helps students with academics, courses, deadlines, departments, campus life & administrative queries*

---

# 1. Problem Scenario

A large university wants an AI assistant that can:

- Help students navigate:
  - courses
  - academic deadlines
  - grading policies
  - campus rules
  - departments & contacts
  - fees, scholarships
  - placement guidelines
- Retrieve relevant information using RAG
- Provide personalized guidance based on student program & year
- Integrate with university systems using tools
- Help with course selection
- Provide timetable queries
- Guide international students
- Handle repeated questions efficiently
- Maintain context about student identity and program

The system must operate like **an intelligent, always-on campus helpdesk**, using:

- **Ollama + Mistral**
- **LangChain**
- **RAG**
- **Streamlit UI + FastAPI backend**
- **Tools for timetable, policies, fees, course info**
- **Memory**
- *(Bonus)* Agentic reasoning, caching, autonomous routing

---

# 2. Functional Requirements

## A. Core Capabilities

1. Multi-turn guidance conversations
2. Retrieve university-specific information
3. Handle student identity and program-specific responses
4. Provide campus navigation help
5. Help with:
    - Course prerequisites
    - Program structure
    - Fee breakdown
    - Exam schedules
    - Academic timeline
6. Retrieve documents via RAG (policies, rules, guides)
7. Personalize advice for:
    - B.Tech
    - MBA
    - PhD
    - Exchange students

---

# 3. RAG Requirements

### The RAG dataset should include:

- Course catalog
- Program structures
- Academic calendars
- Fee rules & scholarship policies
- Hostel guidelines
- Placement preparation guidelines
- Department information
- Student handbook (PDF)
- Library rules
- Lab safety documents
- International student guide

Vector store:

- ChromaDB or FAISS
  Students must justify clearly.

### Expected RAG Outcomes:

- Course descriptions
- Deadlines
- FAQs
- Administrative steps

---

# 4. Memory Requirements

### ✔ Entity Memory

Store:

- Student name
- Student program (MSc, BTech, MBA)
- Year (1st, 2nd, final year)
- Department
- Ongoing issues
- Courses student is interested in

### ✔ Summary Memory

Keep long advising conversations manageable.

### ✔ Optional: Long-term Vector Memory

Store:

- previously asked doubts
- academic interests
- preferred electives

---

# 5. Tools / Functions Students Can Build

Students must implement **4–7 tools**.

---

## 1. Academic Calendar Tool

Given:

- academic year
- program

Returns:

- deadlines
- exam dates
- holidays

- registration window

Data source:

- Static JSON or CSV files

---

## 2. Course Lookup Tool

Input:

- course code (e.g., AI101)
- program

Returns:

- prerequisites
- credits
- syllabus topics
- lecture timings

Dataset:

- course catalog (CSV, PDF, or scraped text)

---

## 3. Timetable Tool

Input:

- student year
- department

Output:

- weekly timetable
- classroom numbers
- lab schedule

---

## 4. Fee Structure & Scholarships Tool

Input:

- program

- nationality (domestic/international)

Output:

- fee breakdown
- scholarship eligibility
- payment deadlines

---

## 5. Faculty Directory Tool

Input:

- department name

Output:

- list of faculty
- email
- office location
- consultation hours

---

## 6. Campus Map Tool (Optional)

Integrate with:

- Google Maps
- OpenStreetMap

Returns:

- location of departments
- walking directions
- cafeteria locations

---

## 7. Student Support Tool (Optional)

Handle:

- hostel queries
- medical center info
- library timings
- counseling center

---

# 6. APIs Students May Use (Optional)

### Map APIs

- Google Maps: https://developers.google.com/maps/documentation
- OpenRouteService: https://openrouteservice.org/dev/#overview

### University Data Sources

Students generate:

- static CSV
- PDFs
- JSON

---

# 7. Agentic AI Design (Bonus)

### Agent Roles

1. **Intake Agent**
   - Understand if student has academic, administrative, or campus-life query
2. **Academic Guidance Agent**
   - Helps with program structure, prerequisites, electives
3. **RAG Retrieval Agent**
   - Fetches academic guidelines, deadlines
4. **Administrative Agent**
   - Handles fees, scholarships, hostel, rules
5. **Campus Navigation Agent**
   - Decides when direction/map tools are needed

### Autonomous Logic Ideas

- If student asks “Can I take ML as an elective?” → auto-check prerequisites
- If semester registration is approaching → give reminders
- If student asks “Where is my exam?” → auto-run timetable tool

---

# 8. Caching Strategy (Bonus)

Cache:

- course catalog responses
- timetable entries
- fee rules
- campus policies

Why?

- These are frequently accessed
- Policies rarely change
- Reduces API/tool calling time

---

# 9. ASCII System Architecture Diagram

Copy code

```
                    ┌──────────────────────────────┐
                    │          Streamlit UI        │
                    │  (Chat + Timetable + Maps)   │
                    └──────────────────────────────┘
                                   │
                                   ▼
                          ┌────────────────────┐
                          │      FastAPI API   │
                          └────────────────────┘
                                   │
        ┌──────────────────────────┼──────────────────────────┐
        │                          │                          │
        ▼                          ▼                          ▼
┌──────────────┐            ┌────────────────┐          ┌──────────────────────┐
│  LLM (Mistral)│            │   RAG Engine   │          │   University Tools   │
└──────────────┘            │(FAISS/ChromaDB)│          │ (Calendar, Courses, etc)│
                            └────────────────┘          └──────────────────────┘
                                   │
                                   ▼
                          ┌────────────────────┐
                          │     Vector Store   │
                          └────────────────────┘
```

## 10. Folder Structure

```
university-assistant/
│
├── backend/
│   ├── main.py
│   ├── tools/
│   │     ├── calendar.py
│   │     ├── course_lookup.py
│   │     ├── timetable.py
│   │     ├── fees.py
│   │     ├── faculty.py
│   │     └── campus_map.py
│   ├── rag/
│   │     ├── vector_store.py
│   │     ├── retriever.py
│   │     └── embedder.py
│   ├── memory/
│   └── models/
│
├── frontend/
│   └── app.py (Streamlit)
│
├── evaluation/
│   ├── rag_eval.ipynb
│   ├── bleu_rouge.ipynb
│   ├── latency_tests.ipynb
│   └── advising_quality_tests.ipynb
│
└── data/
    ├── course_catalog/
    ├── academic_calendar/
    ├── policies/
    └── student_handbook/
```

# 11. Evaluation Tasks

## A. RAG Evaluation

Queries:

- “What are prerequisites for DS201?”
- “When does registration close?”

Metrics:

- Precision@k
- Recall@k
- Hit rate

---

## B. Advising Quality Scoring

Evaluate:

- Relevance
- Correctness
- Personalization
- Non-hallucination
- Consistency with university policy

---

## C. Response Quality Tests

- ROUGE-L
- BLEU
- BERTScore

---

## D. Latency Tests

Measure:

- timetable query time
- course lookup speed
- RAG retrieval
- LLM response

---

## E. Stress Tests

Simulate:

- 200 students using same endpoint
- large PDF handbook ingestion
- long multi-turn advising chats

---

# 12. Deployment Requirements

## Streamlit UI:

Must include:

- Chat interface
- Course lookup UI
- Academic timeline display
- Timetable viewer
- Campus map integration
- Fee breakdown cards

## FastAPI Backend:

Endpoints:

- `/chat`
- `/course`
- `/timetable`
- `/fees`
- `/calendar`
- `/rag`
- `/faculty`

---

# 13. Student Submission Checklist

Students must include:

✔ RAG system
✔ Streamlit UI
✔ FastAPI backend
✔ Course/fees/timetable tools
✔ Memory & caching justification
✔ Evaluation notebook
✔ Architecture diagram
✔ Example student conversations
✔ Running instructions